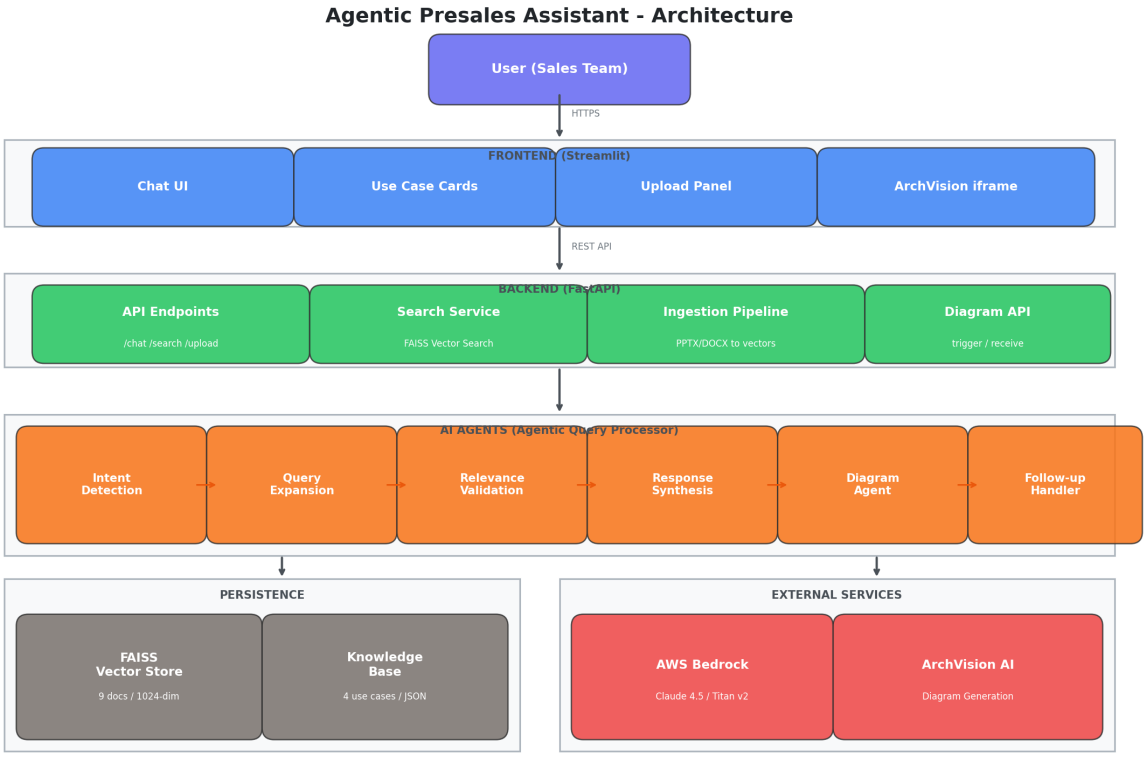
Agentic Presales Assistant

Scope

Sales teams at Ericsson spend significant time searching through scattered documents (PowerPoints, Word files, PDFs) across SharePoint and shared drives to find relevant GenAI/Agentic AI use cases before customer engagements. There is no single intelligent interface to quickly match a customer's specific challenge to available solutions.

Build an AI-powered Presales Assistant chatbot that enables sales teams to:
- Find relevant use cases instantly using natural language queries
- Get business-ready synthesized responses for customer conversations
- Generate architecture diagrams on demand via integration with ArchVision AI
- Self-service upload new use cases without CLI or technical knowledge

Architecture



Frontend — Streamlit (Python 3.10, Port 8501)

app_enhanced.py as the main application with a conversational chat interface supporting natural language queries, conversation history with session memory, and rich response rendering with markdown, business cards, and Mermaid diagrams. Sidebar displays Knowledge Base stats (total solutions, domain breakdown). Quick Action buttons for one-click exploration (Explore OSS/BSS, Compare Solutions, Show All Use Cases). Embedded ArchVision AI iframe for on-demand architecture diagram generation with 3-second auto-polling for callback response. Collapsible Upload Panel accepting PPTX /DOCX/PDF files for self-service use case ingestion. Agent Thinking Process expandable section showing step-by-step reasoning transparency.

Backend — FastAPI (Python 3.10, Port 8000)

main.py as the main application with all REST endpoints (/chat, /search, /use-cases, /upload, /trigger-diagram, /receive-diagram, /get-diagram), CORS middleware for cross-origin access, and service initialization. **search_service.py** for FAISS vector search with metadata enrichment, score-based ranking, and use case listing from processed JSON files. **llm_service.py** as the AWS Bedrock Claude wrapper handling all LLM invocations with configurable region and model via environment variables. **agentic_processor.py** as the core orchestrator (380 lines) managing the full agentic query pipeline. diagram_agent.py for Mermaid architecture and flow diagram generation with styled components and color-coded nodes.

AI Agent Layer — AWS Bedrock (Claude Sonnet 4.5)

6 agents implemented across the query processing pipeline.

Intent & Planning stage has the Intent Detection agent classifying queries into 5 types (simple_search, comparison, exploration, brief_request, clarification_needed) and generating multi-step execution plans with domain/tech filters.

Search & Validation stage runs 3 agents in sequence — Query Expansion agent adds telecom domain context to vague queries, FAISS Semantic Search executes 1024-dim cosine similarity matching, and Relevance Validation agent uses LLM to judge result relevance returning strict JSON with false-positive filtering.

Synthesis stage has the Response Synthesis agent generating intent-specific business-ready answers, and the Format Decision agent choosing output type (summary/technical/business/cost) with optional Diagram Agent triggering for Mermaid architecture visualizations.

Follow-up Handler detects conversation continuations, classifies follow-up type (technical/business/cost/detailed), retrieves previous context from conversation history, and generates deeper responses with architecture diagrams when appropriate.

Persistence / Storage

FAISS vector index (faiss_index.index + faiss_index.pkl) with 9 indexed document chunks across 1024 dimensions, supporting sub-5ms cosine similarity search.

Processed data as JSON files per use case containing structured fields (title, problem, solution, technical_details, domain, tech_type, raw_content) with separate chunk and embedding files for each use case.

In-memory diagram store (Python dict) keyed by session_id for ArchVision AI callback responses, supporting base64 JPEG, image URL, and Mermaid code formats.

Kubernetes ConfigMaps for code delivery and data persistence across pod restarts, with emptyDir volumes for runtime uploads.

External Services

ArchVision AI (archvision-ai.hahn054.rnd.gic.ericsson.se) with bidirectional integration — outbound via parameterized URL (?session=<id>&context=<text>&callback=<endpoint>) loaded in iframe, inbound via POST callback receiving base64 JPEG diagrams with session_id matching.

AWS Bedrock (us-east-1) as the LLM and embedding provider — Claude Sonnet 4.5 (us.anthropic.claude-sonnet-4-5-20250929-v1:0) for all agentic reasoning (3-5 calls per query), Titan Embed Text v2 (amazon.titan-embed-text-v2:0) for 1024-dimensional document and query embeddings. Permanent IAM credentials (no session token rotation required).

Nginx Ingress Controller on EWS cluster HAHN054 providing HTTPS access via wildcard DNS (*.hahn054.rnd.gic.ericsson.se) with websocket support annotations for Streamlit compatibility.

Access URLs

****UI:**** https://aif-chatbot.hahn054.rnd.gic.ericsson.se/
****API:**** https://aif-chatbot-api.hahn054.rnd.gic.ericsson.se/

API Docs: https://aif-chatbot-api.hahn054.rnd.gic.ericsson.se/docs
ArchVision AI: https://archvision-ai.hahn054.rnd.gic.ericsson.se/

Access Requirements:
- Ericsson internal network (VPN or office)
- No login or credentials required
- HTTPS only (accept self-signed certificate on first visit)

Use Cases

UC1 — Find relevant AI solutions via natural language query
A sales team member types a customer challenge in plain English (e.g. "CSP wants to reduce integration time"). The Agentic Query Processor detects intent, expands the query with telecom context (multi-vendor OSS/BSS, API orchestration, automated testing), performs semantic vector search across the knowledge base, validates relevance via LLM, and synthesizes a business-ready response with matching use case cards showing domain, technology, and match score.

UC2 — Explore and compare available solutions
The user clicks "Compare Solutions" or asks "What solutions are available?". The system retrieves all indexed use cases with full details (problem, solution, value proposition), presents them in expandable cards grouped by domain (OSS/BSS, Network), and generates a comparative summary highlighting key differences in approach, technology, and applicability.

UC3 — Generate architecture diagrams on demand
After receiving a chat response, the user clicks "Create Architecture Diagram". The system triggers ArchVision AI via iframe with contextual data (use case title, solution description, session ID, callback URL). The user interacts with the diagram editor, clicks "Return to Bot", and the generated architecture diagram (base64 JPEG) is automatically received via callback and displayed inline in the chatbot with a download button.

UC4 — Upload and ingest new use cases via self-service
A user expands the "Upload New Use Case" panel, provides a name, and drops PPTX/DOCX/PDF files. The system saves files to the knowledge base folder, runs the ingestion pipeline (MarkItDown conversion structure extraction chunking Titan v2 embedding FAISS indexing) , and makes the new use case searchable immediately — no CLI, no redeployment required.

UC5 — Conversational follow-up with context memory
After an initial response about ADIF, the user asks "Tell me more about the architecture". The Follow-up Handler detects conversation continuity, retrieves the previous use case from history, classifies the follow-up type as "technical", generates a detailed architecture response with component breakdown, and triggers the Diagram Agent to produce a Mermaid flowchart showing the multi-agent system design.

UC6 — Vague query handling with intelligent expansion
A user types a broad query like "reduce costs". The Query Expansion agent enriches it with telecom-specific context (OpEx reduction, OSS/BSS automation, operational efficiency, AI-driven optimization). The Relevance Validation agent then strictly filters results to only those genuinely addressing cost reduction, preventing false positives from generic keyword matches.

Demo Link

AIF_chatbot.mp4

Presentation link